# Generalized Prediction of Task Success in Generative Language Models

Steven Woodrum (leader)
*Dept. of Computer Science*
*Virginia Tech*
Alexandria, VA USA
swoodrum@vt.edu

Dharaneesh Coimbatore Kandasamy Venkatasalam
*Dept. of Computer Science*
*Virginia Tech*
Alexandria, VA USA
dharaneesh@vt.edu

Nick York
*Dept. of Computer Science*
*Virginia Tech*
Alexandria, VA USA
nickyork@vt.edu

Nishan Pokharel
*Dept. of Computer Science*
*Virginia Tech*
Alexandria, VA USA
nishan98@vt.edu

***Abstract*—This document describes our proposed project: an AI evaluator that predicts the probability that a given AI system answers a question correctly without actually running it. The motivation for this project stems from the need for AI system evaluators that can be trusted in order for organizations such as schools to understand their systems' capabilities for given tasks. The project will be facilitated by the 2026 Stanford AIMS Predictive AI Evaluation Competition. Our proposed approach builds on existing work by training a model built from two neural networks that estimate a system's ability and a question's difficulty using text embeddings of the provided database of system information and prompts. Our approach will be evaluated by the competition hosts on a set of unseen tasks using the Brier ALC score. We expect our final product to outperform a standard baseline neural network that we will initially develop. Additionally, we hope to use test-time adaptation to further improve performance based on a small number of real outcomes provided by the competition test harness at test time.**

***Index Terms*—component, formatting, style, styling, insert**

## I. Introduction

Many organizations, including schools, hospitals, and businesses, increasingly need to assess the suitability of AI systems for their intended use cases, but lack the extensive resources currently required. Our project has the goal of designing a reliable evaluator that can be used to predict the probability that a given AI system answers a specific question or task correctly, using only descriptions of the system's properties, the question to be answered, and optionally several sample question-response pairs generated by the system, all without explicitly running the AI system on the target question. The 2026 Stanford AIMS Predictive AI Evaluation Competition provides data, a concrete objective, an evaluation service, and a leaderboard to further motivate this project, and evaluates predictive capacity by testing submitted models on held-out benchmarks [1].

AIMS competition organizer Sang Truong has published multiple papers motivating the topic. One such paper describes how public benchmarks play a significant role in steering LLM development, and highlights the issue that fundamental information limitations mean evaluators are only able to cover a subset of socially relevant capabilities with their evaluation tasks [2]. The authors propose randomized evaluation mechanisms as a way to incentivize model developers to train to cover a broader set of socially relevant tasks. Our project complements this by developing a system that can evaluate on unknown inputs. Another paper compares and contrasts student evaluation and AI evaluation and provides an overview on the struggles of reliably measuring capability change over time [3]. It proposes a roadmap that connects strengths through a unified framework organized around static measurement (where ability is inferred at a fixed point in time) and dynamic measurement (where ability is inferred as a trajectory). The main goal is stated to be to accelerate progress on the common problem of evaluating intelligent systems. Our project relates to this idea by taking Item Response Theory, a tool typically used to evaluate student test-takers, and applying it to AI models, using benchmarks as the test banks.

## II. Related Work

Zhou et al. introduce PSN-IRT, a neural Item Response Theory framework for analyzing large language model benchmarks. It uses separate feed-forward networks to estimate a model's scalar ability and a question's difficulty, discrimination, guessing rate, and feasibility. The authors use these estimates to examine benchmark quality and construct smaller evaluation sets [4].

Zintgraf et al. propose CAVIA, a meta-learning method that separates shared model parameters from a small set of task-specific context parameters. During meta-training, the shared parameters are optimized so that the context parameters can be updated from only a few examples, while at test time only the context parameters are changed. This provides a useful foundation to leverage test-time adaptation stage because it limits adaptation to a small ability correction instead of updating the entire predictor [5].

## III. Proposed Approaches

We propose a deep neural predictor architecture that estimates the probability that an AI system, under a particular configuration, will successfully answer a question. We will implement three approaches with each adding to the previous design.

Approach 1 will be a basic binary classifier. Encoded system metadata and question features will be combined and

passed through a feed-forward neural network (FNN), with a single sigmoid output representing the probability of success. The network will be trained using binary cross-entropy and backpropagation.

Approach 2 will adapt PSN-IRT using separate subject and item networks as developed in [4]. However, the subject network will produce an ability vector instead of a scalar, while the item network will produce a discrimination vector and difficulty offset. Their interaction will lastly pass through a sigmoid activation to predict success. We will use system metadata and question representations instead of identifier-only inputs to support prediction on unseen questions, and omit the original guessing and feasibility parameters to simplify estimation. Both networks will be trained jointly using binary cross-entropy. The ability dimensions will be learned latent factors rather than predefined skills.

Approach 3 will add CAVIA-style meta-learning and test-time adaptation to Approach 2. During offline training, each episode will contain separate support and query questions for a subject–benchmark task. Support outcomes will update a small ability correction vector, while query loss after adaptation will train the shared networks through that update. At test time, the networks will remain fixed and only the correction will change using revealed support outcomes. This will test whether training specifically for adaptation improves predictions beyond the base predictor.

## IV. System Design

### A. Architecture

Fig. 1 shows the complete system with CAVIA-style adaptation. System metadata will be encoded into subject features, while question text will be embedded using a frozen pretrained text encoder and combined with permitted item metadata. Text representations will be cached, with long prompts processed in chunks. The subject FNN will produce the initial ability vector $\theta_0$, and the item FNN will produce discrimination $a_i$ and difficulty $b_i$. The final prediction will be:

$$P(correct) = sigmoid[a_i^T(\theta_0 + \delta) - b_i]$$

The correction $\delta$ will start at zero and will be updated from revealed outcomes during adaptation. It will reset for each new subject–benchmark evaluation. Network sizes and other hyperparameters will be optimized using validation data.

### B. Languages and Software Tools

Python will be the primary implementation language. PyTorch will be used to construct networks and perform gradient-based adaptation, pandas and NumPy for data preparation, and SBERT for pretrained text representations. Scikit-learn will support pre-processing and evaluation, and Matplotlib will visualize learning curves and calibration plots. Git, model checkpointing, split manifests, and random seeds will support reproducibility.

### C. Data Sets

The project will use the AI Measurement Data Bank, the public training dataset for the 2026 Stanford Predictive AI Evaluation Competition [1]. The current release contains six benchmarks, 12,632 items, 571,921 responses, and 543,735 execution traces. An item represents a question or task, a subject represents an AI system with a specific model and evaluation configuration, and a response records the outcome of an attempt. The proposed method will estimate the probability that a subject successfully completes an item using the subject and item information available through the competition interface. Execution traces will not be used as prediction inputs because they are unavailable at prediction time.

### D. Evaluation

The proposed approaches will be compared against two baselines: a constant predictor that returns 0.5 (random guess) and the competition's empirical-mean predictor, which uses revealed outcomes for the same subject and benchmark [6]. A subset of benchmarks will be held out during pre-competition testing to emulate the competition's unseen-benchmark evaluation technique.

The primary evaluation metric will be Brier Area under Learning Curve (Brier ALC), the official competition metric [1]. In this case, Brier ALC is a weighted average of prediction error across test-time sample budgets of 0, 1, 3, 7, 15, and 31 samples. Lower scores indicate more accurate probability estimates across different amounts of test-time evidence. Expected Calibration Error will be reported to assess whether predicted probabilities match observed success rates. Model size, prediction time, and adaptation time will also be measured to assess the computational cost of the proposed approaches in order to meet secondary competition requirements. Additional experiments will compare scalar and vector abilities, metadata-only and text-only inputs, and ordinary correction updates in Approach 2 against CAVIA-trained adaptation in Approach 3. We will also test temperature scaling fitted only on validation predictions. These comparisons will show whether the structured predictor and adaptation training each improve performance.

## V. Methods

The first iteration of this effort ("Approach 1") had two research objectives: 1) Learn the basics of deep learning modeling and evaluation in Python and 2) Develop a naive feedforward neural network (FNN) with zero-shot inference that is better at the prediction task than randomly guessing. Performance was measured by Brier score over multiple predictions, which reduces to mean squared error (MSE) when estimating the probability of a single class label. Four variants of Approach 1 were built and evaluated. Each team member completed all steps of Approach 1 in parallel to maximize skill-building in the foundational stage of the project.

Variant 1 (Steven): Input consisted of SBERT all-MiniLM-L6-v2 text embeddings of "task content" (the task whose success is being predicted, such as "Find the prime factors

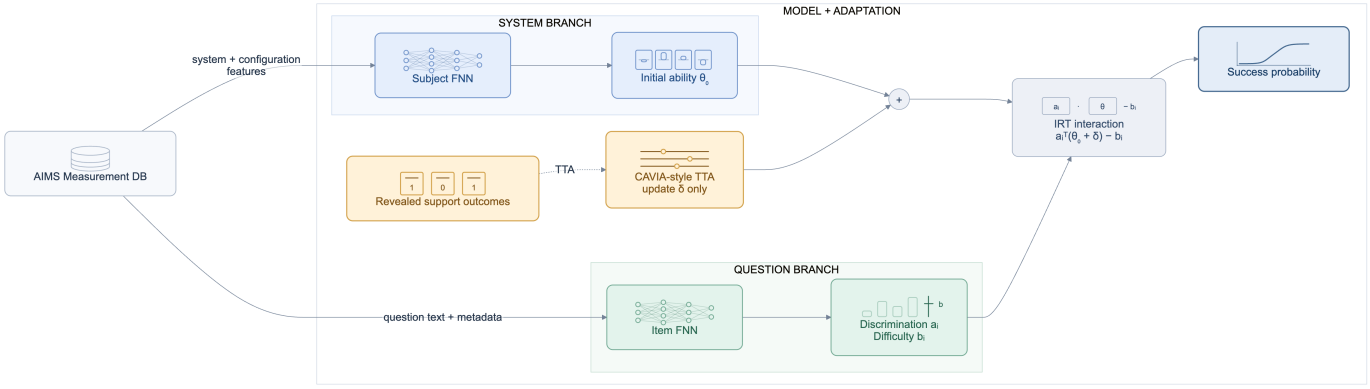



Fig. 1. Proposed system architecture.

of 12,345") concatenated with embeddings of "subject name" (the natural language designation of the model whose performance is being predicted, such as "OpenAI GPT-5.2"). Two hidden layers of 64 and 32 ReLU neurons each learned patterns to infer a single sigmoid output, representing the predicted probability of success. The model was trained and validated on a small sample (N=10k randomly sampled from pool of 572k samples), enabling rapid prototyping.

Variant 2 (Nick): Improved on Variant 1 by increasing training sample size, creating a reduced 16-dimension embedding for each "subject name" with special handling for null entries, and batched training.

Variant 3 (Dharaneesh): Improved on Variant 2 by adding hashed subject model metadata to the input, such as model harness and reasoning effort. Optimized training using Adam, adaptive learning rate, and L2 regularization.

Variant 4 (Nishan): Improved on Variant 3 by chunking inputs to embedding models. Trained, tested, and compared 19 optimization combinations, culminating in top-scoring model using an L2 regularization penalty of 0.001. A comparison of all 19 combinations is given in Fig. 2.

In all variants, the "matharena" and "MMDocRAG" training data was dropped because its labels are non-binary and represent a partial-credit grade instead of a probability of correctness, so do not match our training paradigm.

## VI. Results

The results presented are subdivided by each held-out benchmark under test (e.g., swebench, matharena) in a leave-one-benchmark-out cross-validation. However, the results of Approach 1 architectures are only meaningful in aggregate for two reasons: 1) benchmark names are anonymized during test and 2) Approach 1 does not explicitly condition on benchmark name or category, meaning, at best, hidden learned representations of item content loosely correlate task category with P(success). Instead, results are assessed in aggregate to measure stability (performance cross-validated across benchmarks) and generalization (performance at test time on unseen benchmarks). Table I shows results per held-out benchmark, and Table II shows aggregate results.

The progressive improvement of each variant using techniques learned early in the semester suggests that the performance ceiling of the zero-shot naive FNN approach may still be improved with tuning. However, there are critical caveats that cast doubt on the overall viability of the approach. A robust model should perform similarly during validation and test, but surprisingly, the Brier scores improved by 4.2 and 11.3 percentage points in the top two models during test. Evidence suggests the outsized performance is largely due to luck: the author noted a high degree of sensitivity – variability between training/testing runs – and the test variances of the two best performing models are an order of magnitude greater than the others.

There are multiple opportunities for improvement. Two of the training benchmarks, "matharena" and "MMDocRAG", were unused in Approach 1 due to qualitative differences in their labels. Sub-networks or rule-based transformations of those benchmarks' data may incorporate them into the next training corpus, exposing future models to a greater breadth of subject matter. For example, the partial-credit grades used as labels for "matharena" could be interpreted as probabilities, or rounded up to 1 or down to 0 - two primitive but valid techniques. However, the competition organizers test submissions on additional, hidden held-out benchmarks, making it unlikely the complete training corpus would provide adequate coverage.

A new architecture is required that leverages powerful white-listed generative AI models, augments training data using smaller, ancillary models, or performs test-time adaptation (TTA). Approach 3 will incorporate ancillary models and test-time adaptation, and is described in the Methods section of this report.

## VII. Acknowledgments

OpenAI GPT-5.6 and Claude Opus 5 generative AI models were used to summarize references identified during the literature review. The proposed system architecture is a novel contribution of the authors. Various AI coding assistants were used.

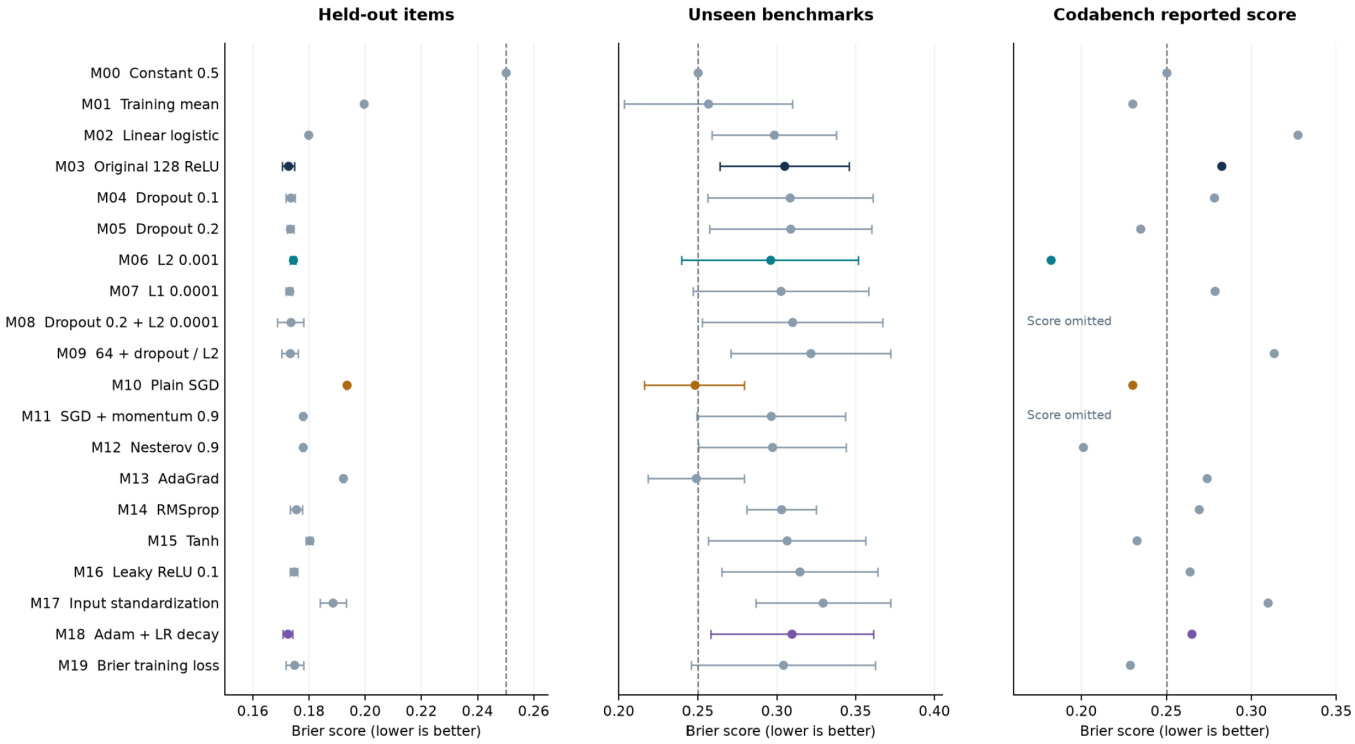



Fig. 2. Results of Approach 1 Variant 4 permutations. Left plot is Brier scores for item-held-out evaluation, center plot for benchmark-held-out evaluation, and right plot for competition performance.

## REFERENCES


[1] "The Predictive AI Evaluation Competition at NeurIPS 2026," https://aimslab.stanford.edu/competition, 2026.

[2] S. T. Truong, S. L. Wang, N. Haber, and S. Koyejo, "Strategic evaluation: Incentivizing ai capability coverage with private benchmarks," 2026. [Online]. Available: https://api.semanticscholar.org/CorpusID:290281952

[3] S. T. Truong, N. D. Goodman, E. Brunskill, B. Domingue, N. Haber, and S. Koyejo, "A measurement science roadmap: From human assessment to ai evaluation," 2026. [Online]. Available: https://api.semanticscholar.org/CorpusID:288249472

[4] H. Zhou, H. Huang, Z. Zhao, L. Han, H. W. Wang, K. Chen, M. Yang, W. Bao, J. Dong, B. Xu, C. Zhu, H. Cao, and T. Zhao, "Lost in benchmarks? rethinking large language model benchmarking with item response theory," in *AAAI Conference on Artificial Intelligence*, 2025. [Online]. Available: https://api.semanticscholar.org/CorpusID:278782250

[5] L. M. Zintgraf, K. Shiarlis, V. Kurin, K. Hofmann, and S. Whiteson, "Fast context adaptation via meta-learning," in *International Conference on Machine Learning*, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:59413758

[6] "Empirical mean predictor," https://github.com/aims-foundations/paiec_baseline/blob/main/empirical_mean/README.md, 2026.


## APPENDIX A
## SCHEDULE AND TASK ASSIGNMENTS

Table III indicates each team member's primary responsibilities. Table IV assigns tasks for the entire 13-week project.

TABLE I
APPROACH 1 LEAVE-ONE-BENCHMARK-OUT RESULTS

| Variant | multi_swebench LOBO Brier score | real_webagents LOBO Brier score | researchcodebench LOBO Brier score | swe_rebench LOBO Brier score |
|---|---|---|---|---|
| **1 (Steven)** | 0.3317795 | 0.309318 | 0.329360902 | 0.413426 |
| **2 (Nick)** | 0.3104 | 0.2839 | 0.3113 | 0.2751 |
| **3 (Dharaneesh)** | 0.257 | 0.249 | 0.284 | 0.347 |
| **4 (Nishan)** | 0.27917 | 0.27299 | 0.25312 | 0.37802 |

TABLE II
APPROACH 1 AGGREGATE RESULTS

| Variant | validation avg Brier | validation Brier variance | test avg Brier ↓ | test Brier variance |
|---|---|---|---|---|
| **1 (Steven)** | 0.345971 | 0.002124 | **0.27624** | 0.001026 |
| **2 (Nick)** | 0.295175 | 0.000341 | **0.281292** | 0.005032 |
| **3 (Dharaneesh)** | 0.28425 | 0.001974 | **0.238466** | 0.029954 |
| **4 (Nishan)** | 0.295825 | 0.003126 | **0.18214** | 0.016137 |

TABLE III
TEAM ROLES

| Team member | Role |
|---|---|
| Dharaneesh | All aspects in parallel |
| Nick | All aspects in parallel |
| Nishan | All aspects in parallel |
| Steven | All aspects in parallel |

TABLE IV
SCHEDULE AND TASK ASSIGNMENTS*

| | |
|---|---|
| Week 1 (13-19 September 2026) | |
| Register team for competition | Steven |
| Analyze competition data sets | Dharaneesh |
| Research and draft abstract, introduction, related work | Nick |
| Research and draft proposed approaches and system design | Nishan |
| Project website | Steven |
| Finalize proposal | Steven |
| Week 2 (20-26 September) | |
| *Proposal due 22 Sep* | |
| Submit proposal package | Steven |
| Reserve long-lead services (compute, recording studio, etc.) | Steven |
| Exploratory data analysis | All |
| Split data into train/validate/test | Dharaneesh |
| Data pre-processing and retrieval infrastructure | Dharaneesh |
| Confirm access to cloud compute for training/testing | Nick |
| Clarify expectations for project website and visualizations | Steven |
| Design "Approach 1" on paper | All |
| Week 3 (27 September - 3 October) | |
| Review Approach 1 design with teaching staff | All |
| Implement, train, and internally test Approach 1 | All |
| Week 4 (4-10 October) | |
| Submit Approach 1 to competition and analyze feedback | All |
| Document Approach 1 results for Checkpoint I report | All |
| Design "Approach 2" iteration on paper | All |
| Week 5 (11-17 October) | |
| Finalize Checkpoint I report | Steven |
| Review Approach 2 design with teaching staff | All |
| Implement, train, and test Approach 2 | All |
| Week 6 (18-24 October) | |
| *Checkpoint I due 20 Oct (paper update)* | |
| Submit Checkpoint I report | Steven |
| Submit Approach 2 to competition and analyze feedback | All |
| Document Approach 2 results for Checkpoint II report | All |
| Design "Approach 3" iteration on paper | All |
| Week 7 (25-31 October) | |
| *Stanford AIMS Competition deadline 30 Oct* | |
| Finalize Checkpoint II report | Steven |
| Review Approach 3 design with teaching staff | All |
| Implement, train, and internally test Approach 3 | All |
| Week 8 (1-7 November) | |
| Submit Approach 3 to competition and analyze feedback | All |
| Document Approach 3 results for final report | All |
| Week 9 (8-14 November) | |
| *Checkpoint II due 10 Nov (paper update)* | |
| Submit Checkpoint II report | Steven |
| Write concluding sections of final report | All |
| Week 10 (15-21 November) | |
| *Original AIMS Competition deadline; Eval site closes 20 Nov* | |
| Review conclusions with teaching staff | All |
| Finalize final report | Steven |
| Week 11 (22-28 November) | |
| *Thanksgiving Break* | |
| Write oral presentation slides * All | |
| Write 1-page summary | All |
| Week 12 (29 November - 5 December) | |
| Practice oral presentation | All |
| Record and edit video | All |
| Week 13 (6-11 December) | |
| *Final Presentations; Final Project Package due 11 Dec* | |
| Present oral presentation to class | All |
| Submit final project package | Steven |
| ** In later weeks, tasks will be subdivided and assigned to individuals.* | |